

Neural Networks for Runtime Verification

Alan Perotti, Artur d'Avila Garcez and Guido Boella

Abstract—A recent trend in High-Performance Computation is parallel computing, and the field of Neural Networks is showing impressive improvements in performance, especially with the use of GPU accelerators. In this paper, we use neural networks to improve the performance of Runtime Verification. Runtime verification is used in a variety of domains -from policy enforcement to electronic fraud detection- to automatically check whether a system meets a temporal specification, by observing the output of the system. In this paper, we present a novel run-time monitoring system, RuleRunner, and we exploit results from the Neural-Symbolic Integration area to encode it in a recurrent neural network. The results show that neural networks can perform real-time online runtime verification. Performance was improved by the parallel architecture and the matrix-based implementation with GPU.

I. INTRODUCTION

Every day, we create 2.5 quintillion bytes of data - so much that 90% of the data in the world today has been created in the last two years alone [1]. This data comes from everywhere: sensors used to gather climate information, posts to social media sites, digital pictures and videos, purchase transaction records, and cell phone GPS signals to name a few. The range of systems and devices generating and storing this data is also wide, and, in the *Internet of Things* scenario [2], it includes any natural or man-made object that can be assigned an IP address and provided with the ability to transfer data over a network. It is essential to continuously monitor the execution of these system: well-timed fault detection can prevent a number of undesired malfunctions and breakdowns, spanning from a credit card fraud to an electric kettle overheating and catching fire. Moreover, the monitoring task has to be performed at run-time and in an efficient way, either when dealing with big data or when embedded in systems with limited computational resources. The field of runtime verification deals with the problem of automatically checking whether a system meets a given temporal specification, analysing a system's output (*trace*) rather than building a model of its internal structure (as in Model Checking). Therefore, runtime verification suits scenarios where the observed system is not known (logs taken from repositories), not accessible (closed source), but also not a software (logs from the activity of office employees).

A recent trend in the field of high-performance computation is parallel computing, and the field of neural network is showing impressive improvements in performance,

especially with the use of GPU accelerators. For example, in 2012 Google used approximately 1,000 CPU-based servers, or 16,000 CPU cores, to develop its neural network (1.7 billion parameters), which taught itself to recognize cats in a series of YouTube videos [3]. One year later, the Stanford team created an equally large network with only three servers using NVIDIA GPUs to accelerate the processing of the big data generated by the network [4].

Our research question is the following:

- *Is it possible to use the connectionist paradigm and the parallel computing techniques in order to improve the performance of a runtime verification system?*

In this paper, we present a novel run-time monitoring system, RuleRunner. RuleRunner creates and maintains a single, detailed state, avoiding the *possible worlds* branching structure typical of tableaux-based formulations. Furthermore, RuleRunner is designed as a set of rules that can be fired in parallel, rather than in a strict sequence. We show how a RuleRunner system can be translated in an equivalent logic program, in order to exploit results from the Neural-Symbolic Integration area to encode it in a recurrent neural network. Finally, we validate our framework by comparing three implementations based on neural networks, observing how sparse form and GPU computing improve the performance, respectively, in terms of absolute speed and scalability.

Neural networks provide intrinsic learning capabilities: in the Future Work session we introduce two relevant scenarios combining runtime verification and property adaptation, but learning is outside the scope of this paper.

The paper is structured as follows: Section 2 introduces related work; Section 3 analyses our rule-based system, RuleRunner, and Section 4 discusses the translation of a rule system in a neural network. Section 5 describes the monitoring within a neural network and related experiments. Section 7 discusses directions for future work, Section 7 ends the paper final conclusions.

II. RELATED WORK

A. Runtime Verification

Like Model Checking, runtime verification (RV) relates an observed system with a formal property ϕ (often formulated in linear temporal logic LTL [5]), determining the semantics of ϕ while executing the system under scrutiny. The black-box, real-time nature of RV poses additional constraints w.r.t. the general Model Checking area: since the

Alan Perotti and Guido Boella are affiliated with the University of Turin, Italy (email: {perotti, boella}@di.unito.it). Artur d'Avila Garcez is affiliated with City University London (email: aag@soi.city.ac.uk)

inner structure of the observed system is unknown, it is not possible to build an automaton for it, nor to *peek in the future* and use path exploration techniques. An RV module, or monitor, is defined as a device that reads a finite trace and yields a certain verdict [6]. A trace is a sequence of cells, which in turn are lists of observations occurring in a given discrete span of time. Runtime verification may work on finite (terminated), finite but continuously expanding, or on prefixes of infinite traces. A monitor may control the current execution of a system (online) or analyse a recorded set of finite executions (offline). Several RV systems have been developed, and they can be clustered in three main approaches, based respectively on rewriting, automata and rules [7]. There are many semantics for finite traces: FLTL [8], RVLTL [9], LTL3 [10], LTL $\pm$ [11] just to name some. Since LTL semantics is based on infinite behaviours, the issue is to close the gap between properties specifying infinite behaviours and finite traces. In particular, FLTL differs from LTL as it offers two *next* operators (X , $\bar{X}$ in [9], X , W in this paper), called respectively *strong* and *weak* next. Intuitively, the strong (and standard) X operator is used to express with $X\phi$ that a next state must exist and that this next state has to satisfy property ϕ . In contrast, the weak W operator in $W\phi$ says that if there is a next state, then this next state has to satisfy the property ϕ . More formally, let $u = a_0..a_{n-1}$ denote a finite trace of length n . The truth value of an FLTL formula ψ (either $X\phi$ or $W\phi$) w.r.t. u at position $i < n$, denoted by $[u, i \models \psi]$, is an element of $\mathbb{B}$ and is defined as follows:

$$[u, i \models X\phi] = \begin{cases} [u, i+1 \models \phi], & \text{if } i+1 < n \\ \perp, & \text{otherwise} \end{cases}$$

$$[u, i \models W\phi] = \begin{cases} [u, i+1 \models \phi], & \text{if } i+1 < n \\ \top, & \text{otherwise} \end{cases}$$

In particular, since FLTL extends LTL, every LTL formula is a FLTL formula, while $FLTL \setminus LTL$ is the set of formulae where W occurs.

B. Neural-symbolic Integration

The main purpose of a neural-symbolic system is to bring together the connectionist and symbolic approaches exploiting the strengths of both paradigms and, hopefully, avoiding their drawbacks. In [12], Towell and Shavlik presented the influential neural-symbolic system KBANN (Knowledge-Based Artificial Neural Network), a system for rule insertion, refinement and extraction from feedforward neural networks. KBANN served as inspiration for the construction of the Connectionist Inductive Learning and Logic Programming (CILP) system [13]. CILP builds upon KBANN and [14] to provide a sound theoretical foundation for reasoning in artificial neural networks and learning capabilities.

In particular, rules are mapped onto hidden neurons, the preconditions of rules onto input neurons and the conclusion

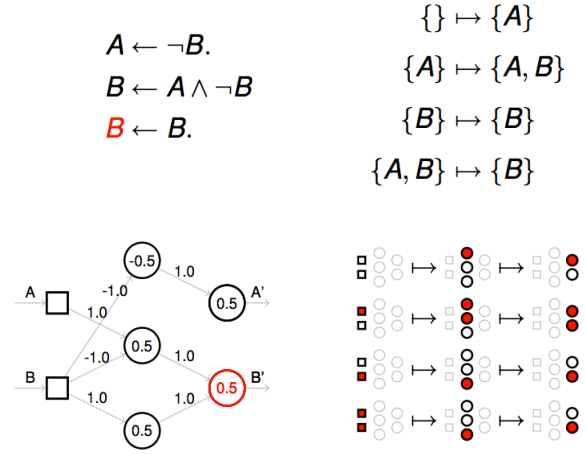


Fig. 1: Logic program as a network (CILP)

of the rules onto output neurons (example in Figure 1). The weights are then adjusted to express the dependence among all these elements.

CILP provides an algorithm to translate a propositional logic program in a neural network that computes the stable model of the initial program. The obtained network implements a massively parallel model for Logic Programming, and it can perform inductive learning from examples, by means of standard learning strategies.

III. RUNTIME VERIFICATION USING A RULE SYSTEM

The first phase of our approach consisted in the design and development of a rule-based runtime monitor, RuleRunner, suitable to be encoded in a neural network. RuleRunner observes finite but expanding traces and returns an FLTL verdict. As it scans the trace, RuleRunner maintains a state composed by rule names, observations and formulae evaluations.

Given a finite set of observations O and a LTL formula ϕ over (a subset of) O , RuleRunner has a state S composed by observations ($o \in O$), rule names ($R[\psi]$) and truth evaluations ($[\psi]V$); $V \in \{T, F, ?\}$ is a truth value. A rule name $R[\psi]$ in S means that the logical formula ψ is under scrutiny, while a truth evaluation $[\psi]V$ means that the logical formula $[\psi]$ currently has the truth value V . The third truth value, $?$, means that it is impossible to give a binary verdict in the current cell: for instance, if the formula is Xa , a has to be observed in the second cell, and it's impossible to evaluate Xa in the first cell.

The state evolves according to rules: RuleRunner is composed by evaluation and reactivation rules (R_E, R_R). Evaluation rules follow the pattern

$$R[\phi], [\psi^1]V, \dots, [\psi^n]V, \rightarrow [\phi]V$$

and, intuitively, their role is to compute the truth value of a formula ϕ under verification, given the truth values of its

direct subformulae ψ^i . Reactivation rules follow the pattern

$$[\phi]? \rightarrow R[\phi], R[\psi^1], \dots, R[\psi^n]$$

and the meaning is that if one formula is evaluated to undecided, that formula (together with its subformulae) is scheduled to be monitored again in the next cell of the trace.

Thus a RuleRunner system is defined as a tuple $\langle R_E, R_R, S \rangle$ (state, evaluation and reactivation rules respectively).

Given a (F)LTL formula ϕ and a trace t , the following algorithms summarise the creation and runtime behaviour of a RuleRunner system monitoring ϕ over t :

Algorithm 1 Construction of a RuleRunner system RR_ϕ

```

1: function RR-BUILD( $\phi$ )
2:   Parse the LTL formula  $\phi$  in a tree
3:   Generate  $R_E$ ,  $R_R$  and the initial state  $S$ 
4:   return  $RR_\phi = \langle R_E, R_R, S \rangle$ 
5: end function

```

Algorithm 2 Runtime verification using RR_ϕ

```

1: function R-MONITOR( $RR_\phi, \text{trace } t$ )
2:   while new observations exist in  $t$  do
3:     Add observations to state
4:     Compute truth values using evaluation rules
5:     if state contains SUCCESS or FAILURE then
6:       return SUCCESS or FAILURE respectively
7:     end if
8:     Compute next state using reactivation rules
9:   end while
10: end function

```

As an example, consider the formula $\phi = a \vee \Diamond b$ and the trace $t = [c - a - b, d - b]$. Intuitively, ϕ means *either a now or b sometimes in the future*. If monitoring ϕ over t , a fails straight from the beginning, while b is sought until the third cell, when it is observed. Thus the monitoring yields a success even before the end of the trace.

In RuleRunner, for first, the formula ϕ is parsed into a tree, with $\vee$ as root and a, b as leaves. Then, starting from the leaves, evaluation and reactivation rules for each node are added to the (initially empty) rule system. In our example, (part of) the rule system obtained from ϕ , namely $RR_{(a \vee \Diamond b)}$, and its behaviour over t are the following:

EVALUATION RULES

- $R[a], a \text{ is not observed} \rightarrow [a]F$
- $R[b], b \text{ is observed} \rightarrow [b]T$
- $R[b], b \text{ is not observed} \rightarrow [b]F$
- $R[\Diamond b], [b]T \rightarrow [\Diamond b]T$
- $R[\Diamond b], [b]F \rightarrow [\Diamond b]?$
- $R[a \vee \Diamond b]B, [a]F, [\Diamond b]? \rightarrow [a \vee \Diamond b]?R$
- $R[a \vee \Diamond b]R, [\Diamond b]T \rightarrow [a \vee \Diamond b]T$
- $R[a \vee \Diamond b]R, [\Diamond b]? \rightarrow [a \vee \Diamond b]?R$
- $[a \vee \Diamond b]T \rightarrow \text{SUCCESS}$

REACTIVATION RULES

- $[\Diamond b]? \rightarrow R[b], R[\Diamond b]$

- $[a \vee \Diamond b]?R \rightarrow R[a \vee \Diamond b]R$

INITIAL STATE

- $R[a], R[b], R[\Diamond b], R[a \vee \Diamond b]B$

EVOLUTION OVER $[c - a - b, d - b]$

state	$R[a], R[b], R[\Diamond b], R[a \vee \Diamond b]B$
+ obs	$R[a], R[b], R[\Diamond b], R[a \vee \Diamond b]B, c$
eval	$[a]F, [b]F, [\Diamond b]?, [a \vee \Diamond b]?R$
react	$R[b], R[\Diamond b], R[a \vee \Diamond b]R$
state	$R[b], R[\Diamond b], R[a \vee \Diamond b]R$
+ obs	$R[b], R[\Diamond b], R[a \vee \Diamond b]R, a$
eval	$[b]F, [\Diamond b]?, [a \vee \Diamond b]?R$
react	$R[b], R[\Diamond b], R[a \vee \Diamond b]R$
state	$R[b], R[\Diamond b], R[a \vee \Diamond b]R$
+ obs	$R[b], R[\Diamond b], R[a \vee \Diamond b]R, b, d$
eval	$[b]T, [\Diamond b]T, [a \vee \Diamond b]T, \text{SUCCESS}$
STOP	PROPERTY SATISFIED

The behaviour of the runtime monitor is the following:

- At the beginning, the system monitors $a, b, \Diamond b$ and $a \vee \Diamond b$ (initial state = $R[a], R[b], R[\Diamond b], R[a \vee \Diamond b]B$). The $-B$ in $R[a \vee \Diamond b]B$ means that both disjuncts are being monitored.
- In the first cell, c is observed and added to the state S . Using the evaluation rules, new truth values are computed: a is false, b is false, $\Diamond b$ is undecided. The global formula is undecided, but since the trace continues the monitoring goes on. The $-R$ in $R[a \vee \Diamond b]R$ means that only the right disjunct is monitored: the system dropped a , since it could only be satisfied in the first cell.
- In the second cell, a is observed but ignored (the rules for its monitoring are not activated); since b is false again, $\Diamond b$ and $a \vee \Diamond b$ are still undecided.
- In the third cell, d is ignored but observing b satisfies, in cascade, $b, \Diamond b$ and $a \vee \Diamond b$. The monitoring stops, signalling a success. The rest of the trace is ignored.

Most of the runtime verification approaches, like RuleR [15], are based on a (potentially exponential) tree-like structure of alternative hypotheses, while RuleRunner is rooted in maintaining a single state composed of the unique truth value of every subformula of the encoded property. Moreover, the *distributed* nature of a RuleRunner state and the local nature of rules (inferring the truth value of a formula from the truth values of its subformulae only) allows different rules to be applied in parallel. Moreover, the fixed number of elements (observations, rule names, truth evaluations) and the implicit representation of time (by means of rule reactivation) allow to build a network with fixed structure and features: this will be explained in the next sections.

A technical report about RuleRunner, including discussions and formal results concerning semantics and complexity, is available on ArXiv [16]; an online tool is hosted at di.unito.it/~perotti/RuleRunner.jnlp.

IV. ENCODING A RULE SYSTEM IN A NEURAL NETWORK

The second phase of our approach is to encode a RuleRunner system in a neural network. This translation is composed by two main steps, respectively encoding a rule system in a logic program and the resulting logic program in a neural network. In this section, we focus on the translation of a RuleRunner rule system in a logic program. CILP provides a sound and complete translation algorithm to encode a logic program into a (standard, feed-forward, recurrent) neural network, thus providing the second step in the general translation process.

The first step of the neural encoding is the translation of a RuleRunner system into an equivalent logic program (Algorithm 3).

Algorithm 3 From RuleRunner to Logic Programs

```

1: function RR2LP( $\phi$ )
2:   Create  $RR_\phi = \langle R_E, R_R, S \rangle$  encoding  $\phi$ 
3:   Create an empty logic program  $LP$ 
4:   for all  $R[\phi], [\psi^1]V, \dots, [\psi^n]V \rightarrow [\phi]V \in R_E$  do  $\triangleright C_E$ 
5:      $LP \leftarrow LP \cup [\phi]V :- \sim[U], R[\phi], [\psi^1]V, \dots, [\psi^n]V$ 
6:   end for
7:   for all  $o \in R_E \cup R_R$  do  $\triangleright C_P$ 
8:      $LP \leftarrow LP \cup o :- \sim[U], o$ 
9:   end for
10:  for all  $R[\phi] \in R_E \cup R_R$  do
11:     $LP \leftarrow LP \cup R[\phi] :- \sim[U], R[\phi]$ 
12:  end for  $\triangleright C_R$ 
13:  for all  $[\phi] \rightarrow R[\phi], R[\psi^1], \dots, R[\psi^n] \in R_R$  do
14:     $LP \leftarrow xLP \cup R[\phi] :- [U], [\phi]?$ 
15:    for all  $R[\psi^i]$  do
16:       $LP \leftarrow LP \cup R[\psi^i] :- [U], [\phi]?$ 
17:    end for
18:  end for
19:   $LP_\phi = LP$ 
20: return  $LP_\phi$ 
21: end function

```

The algorithm creates a single logic program LP ; however, for the sake of explanation, we distinguish three kinds of clauses: evaluation, reactivation and persistence (marked as C_E, C_R, C_P in Algorithm 3). Intuitively, evaluation and reactivation clauses (in LP) mirror, respectively, evaluation and reactivation rules in RuleRunner. Persistence clauses are used to *remember* observations and active rules by explicit re-generation: these clauses follow the pattern $x :- \sim[UPDATE], x$ (shortened to $[U]$ in the algorithm).

Evaluation clauses are obtained from evaluation rules by adding one extra literal in the body, $\sim[UPDATE]$. The reactivation rules are split into several reactivation clauses, one for each literal in the head of the rule; $[UPDATE]$ is added in the body of all these rules. Finally, for all observations and truth evaluations in the rules, a persistence clause is added.

RuleRunner's monitor loop fires the evaluation and reactivation rules in an alternate fashion. We simulate that by introducing the $[UPDATE]$ literal and using it as a switch: when $[UPDATE]$ holds, only reactivation clauses can hold, while when it does not all reactivation rules are inhibited and evaluation and persistence clauses are potentially active. RuleRunner iteratively builds a state, going through partial solutions; in the same way, some sort of clamping is necessary for the partial results to be remembered by the LP. We achieve that by adding persistence clauses: as long as $[UPDATE]$ does not hold, all observations and rule names are re-obtained at each iteration of the logic program. Another way to achieve this persistence is to add the desired observations/rule names as facts: we opted for the former because persistence clauses have a standard structure, while adding facts to the LP correspond to clamping neurons in a neural network.

For instance, in the previous section we provided a simplified rule system for $a \vee \Diamond b$. Such system corresponds to the following logic program, $LP_{(a \vee \Diamond b)}$:

EVALUATION CLAUSES

- $[a]F :- \sim[UPDATE], R[a], \sim a$
- $[b]T :- \sim[UPDATE], R[b], b$
- $[b]F :- \sim[UPDATE], R[b], \sim b$
- $[\Diamond b]T :- \sim[UPDATE], R[\Diamond b], [b]T$
- $[\Diamond b]? :- \sim[UPDATE], R[\Diamond b], [b]F$
- $[a \vee \Diamond b]?R :- \sim[UPDATE], R[a \vee \Diamond b]B, [a]F, [\Diamond b]?$
- $[a \vee \Diamond b]T :- \sim[UPDATE], R[a \vee \Diamond b]R, [\Diamond b]T$
- $[a \vee \Diamond b]?R :- \sim[UPDATE], R[a \vee \Diamond b]R, [\Diamond b]?$
- $SUCCESS :- \sim[UPDATE], [a \vee \Diamond b]T$

REACTIVATION CLAUSES

- $R[\Diamond b] :- [UPDATE], [\Diamond b]?$
- $R[b] :- [UPDATE], [b]?$
- $R[a \vee \Diamond b]R :- [UPDATE], [a \vee \Diamond b]?R$

PERSISTENCE CLAUSES

- $a :- \sim[UPDATE], a$
- $b :- \sim[UPDATE], b$
- $R[a] :- \sim[UPDATE], R[a]$
- $R[b] :- \sim[UPDATE], R[b]$
- $R[\Diamond b] :- \sim[UPDATE], R[\Diamond b]$
- $R[a \vee \Diamond b]B :- \sim[UPDATE], R[a \vee \Diamond b]B$
- $R[a \vee \Diamond b]R :- \sim[UPDATE], R[a \vee \Diamond b]R$

Summarising, we start from a formal property ϕ expressed as an LTL formula, we compute a RuleRunner rule system RR_ϕ monitoring that formula, and then we encode the rule set in a logic program LP_ϕ . The correctness of this translation can be proved by induction on the monitoring process; we omit such proof due to lack of space. We can then exploit the CILP algorithm [13] to translate the logic program LP_ϕ into an equivalent neural network NN_ϕ .

Each clause (c) of LP_ϕ is mapped from the input layer to the output layer of NN_ϕ through one neuron (labelled C) in the single hidden layer of NN_ϕ . We omitted the computation of the parameters; intuitively, the Translation Algorithm from LP_ϕ to NN_ϕ has to implement the following conditions: (C1) the input potential of a hidden

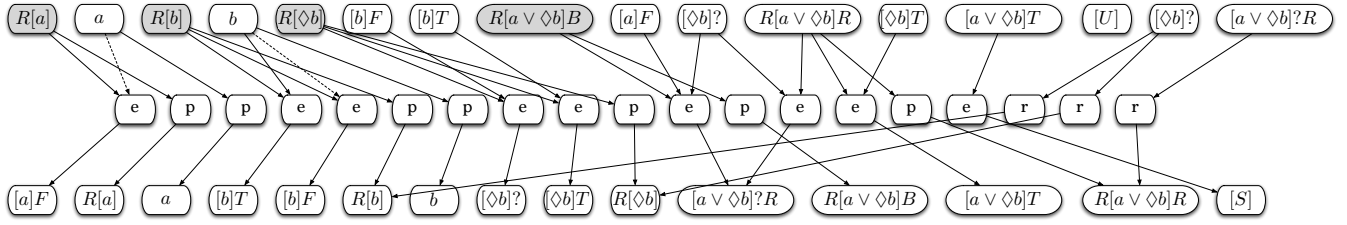


Fig. 2: (simplified) Neural network for monitoring $a \vee \Diamond b$: $NN_{(a \vee \Diamond b)}$

Algorithm 4 Connectionist Inductive Learning and Logic Programming System - Translation

```

1: function  $CIL^2P(LP_\phi)$ 
2:   Create empty network  $NN$ 
3:   for all clause  $c \in LP_\phi$ ,  $c = h :- b_1, \dots, b_n$  do
4:     Add a neuron  $C$  to the hidden layer
5:     If not there, add a neuron  $H$  to the output layer
6:     Connect  $C$  to  $H$ 
7:     for all clause  $b_i \in c$  do
8:       If not there, add a neuron  $B_i$  to the input layer
9:       Connect  $B_i$  to  $C$ 
10:    end for
11:  end for
12:   $NN_\phi = NN$ 
13: return  $NN_\phi$ 
14: end function

```

neuron (C) can only exceed C 's threshold, activating C , when all the positive antecedents of c are assigned the truth value true while all the negative antecedents of c are assigned false; and (C2) the input potential of an output neuron H can only exceed H 's threshold, activating H , when at least one hidden neuron C that is connected to H is activated; the translation algorithm is sound and complete [13].

In our working example, $LP_{(a \vee \Diamond b)}$ is translated into the neural network in Figure 2. The input and output layers include neurons whose labels correspond to atoms in the logic program and the rule system: observations, rule names and truth evaluation. In Figure 2, active neurons are filled in grey, representing the initial state $R[a]$, $R[b]$, $R[\Diamond b]$, $R[a \vee \Diamond b]$. $[U]$ stands for $[UPDATE]$ and $[S]$ for $[SUCCESS]$. Hidden neurons correspond to clauses: for the sake of explanation, each hidden neuron is labelled with e (resp. p, r) to mark that it encodes an evaluation (resp. persistence, reactivation) clause. Solid lines represent connection with positive weights, dashed lines represent negative weights. To simplify the visualisation, all recurrent connections and connections from $[U]$ are omitted in Figure 2. Recurrent links connect neurons in the output layer to the neuron in the input layer with the same label, while there is a positive connection (solid line) from $[U]$ to all r -labeled hidden neurons, and a negative connection (dashed line) from $[U]$ to all e and p -labeled hidden neurons (mirroring the $[UPDATE]$ switch in the logic program). For instance, the leftmost hidden neuron in Figure 2 has ingoing

connections from a (negative weight) and $R[a]$ (positive weight), plus the negative connection from $[U]$ (omitted in the figure); it has an outgoing connection towards $[a]F$: this corresponds to the clause $[a]F :- \sim[UPDATE], R[a], \sim a$, the first evaluation clause in $LP_{(a \vee \Diamond b)}$.

It is worth stressing that our goal is not to achieve better results in terms of monitoring: the monitoring process is exact. As we will describe in the following two sections, the translation of a RuleRunner system in a neural network is done in order to improve performance and exploit learning features.

V. RUNTIME VERIFICATION USING A NEURAL NETWORK

The result of the neural encoding phase is a neural network NN_ϕ able to perform runtime verification, monitoring ϕ over the given trace(s). The general algorithm for neural monitoring is described in Algorithm 5.

Algorithm 5 Runtime Verification using NN_ϕ

```

1: function  $NN\_MONITOR(\phi, \text{trace } t)$ 
2:   Create  $RR_\phi = \langle R_R, R_E, S \rangle$  encoding  $\phi$  (Algorithm 1)
3:   Rewrite  $RR_\phi$  into  $LP_\phi$  (Algorithm 3)
4:   Rewrite  $LP_\phi$  into  $NP_\phi$  (Algorithm 4:  $CILP$ )
5:   Add  $S$  to the input layer
6:   while new observations exist in  $t$  do
7:     Add the new observations to the input layer
8:     Let the network converge
9:     if  $S$  contains SUCCESS (resp. FAILURE) then
10:      return return SUCCESS (resp. FAILURE)
11:    end if
12:    Add  $UPDATE$  to the input layer
13:    Fire the network once
14:  end while
15: end function

```

Adding x to a given layer means activating the neuron corresponding to x in that layer. In terms logic programming, this corresponds to adding the fact x to the program.

It is worth comparing how, from an operational point of view, a RuleRunner system (RR_ϕ) and its neural encoding (NN_ϕ) carry out the monitoring task. In each iteration of the main loop, RuleRunner goes through the list of evaluation rules, adding the result of each active rule to the state. When the end of the evaluation rules list is reached, the reactivation rules are allowed to fire, collecting the output in a new state. In the neural network the alternating

of evaluation and reactivation is achieved by means of the *[UPDATE]* neuron, which acts as a switch. In the evaluation phase, all evaluation rules are fired in parallel until convergence. Moving to the next cell of the trace is achieved by firing all reactivation rules in parallel once.

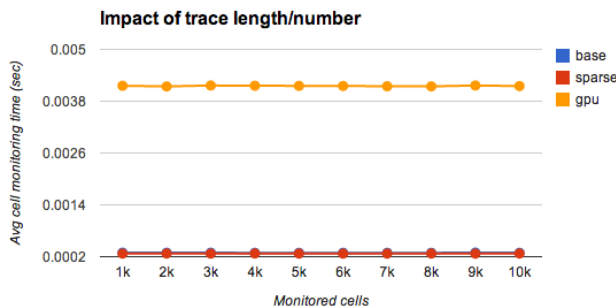
The encoding of RuleRunner in a neural network allows for the monitoring of several traces in parallel. A cell in a trace is a vector of observations which is fed to the neural network. To monitor multiple traces, it is sufficient to compose all observation vectors (of different traces at the same time) as rows in a matrix. The monitoring is then carried out as explained above, with all vector-matrix steps substituted by analogous matrix-matrix operations.

Furthermore, the matrix-based nature of a neural network allows for optimisations in order to speed up the monitoring. We focused on sparse form and GPU computation: the remainder of this section compares these two implementations with the vanilla one and discusses experimental results. In order to make the comparison fair, we used MATLAB 2013a for all tests.

Since we rely on standard MLP with one hidden layer, the size of the neural networks composing NN_ϕ depends solely on the number of neurons in the layers; since NN_ϕ encodes a RuleRunner rule system RR_ϕ , this factor depends on the number of rules in the initial rule system RR_ϕ . However, recall that each rule (clause) only involves truth values of direct subformulae, and it is therefore independent from the size of the whole rule system. Thus the size of the weight matrices depends on the number of rules, but these matrices are very sparse: for instance, since the average number of literals in an evaluation rule is three, the input-hidden weight matrix will have on average three non-zero elements per column. To test the sparse form, we simply used the *sparse*($\cdot$) function, without further alterations of the code w.r.t. the vanilla implementation.

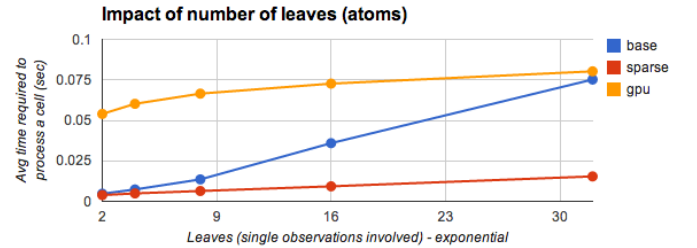
Moreover, we exploited MathWork's Parallel Computing Toolbox to run our code on GPU. We used GPUArray, a special array type with several associated functions that allows to perform computations on CUDA-enabled NVIDIA GPUs directly from MATLAB.

For these tests, the involved matrices represent the encoded property and the fed trace(s). The first batch of tests was run to measure the impact of trace length and number on the average performance time, as depicted below:

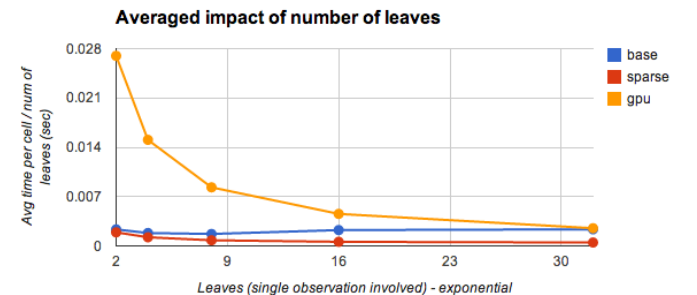


The graph shows the impact of the size and number of the observed traces on the average time required to monitor a single cell. Randomly generated traces were fed to the three implementations; the time required to monitor an increasing number of cells (1000,2000,...,10000) was measured and then averaged. As expected, the nature of the fed traces has no impact on the average time (in seconds) required to monitor a single cell. This preliminary experiment also showed how, for a simple formula ϕ , the *base* (vanilla) and *sparse* implementations run faster and with similar performance, while *gpu* is stably slower.

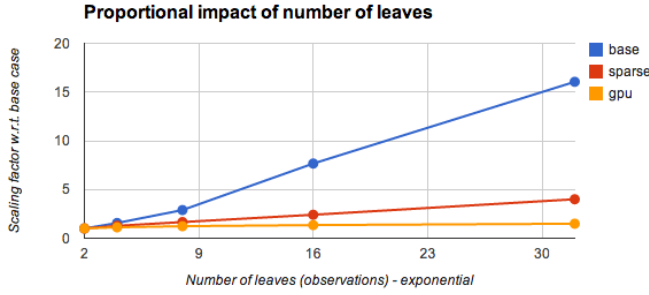
Once we ruled out the traces, we focused on the monitoring network. This network depends solely on the encoded logical formula (the temporal logic property to be monitored). For first, we increased (exponentially) the number of *leaves* in the formula tree. A leaf is a single observation occurring in the formula (so the set of leaves is the terminal alphabet upon which the formula is built). For instance, $\Box(a \vee b)$ has two leaves (a, b), $\Diamond(Xa \wedge ((c \vee d)Ud))$ has four leaves (a, b, c, d) and so on. We tested a standard formula, $\Diamond \bigvee_{i=1}^{n-1} (a_0 \wedge a_i)$, with $n = 2, 4, 8, 16, 32$. For instance, the expanded formula for $n = 16$ (with a_i renamed alphabetically and rewritten in binary form) is $\Diamond((((a \wedge b) \vee (a \wedge c)) \vee ((a \wedge d) \vee (a \wedge e))) \vee (((a \wedge f) \vee (a \wedge g)) \vee ((a \wedge h) \vee (a \wedge i)))) \vee (((a \wedge j) \vee (a \wedge k)) \vee ((a \wedge l) \vee (a \wedge m))) \vee (((a \wedge n) \vee (a \wedge o)) \vee ((a \wedge p) \vee (a \wedge q))))$. We ran these five tests on the three implementations and measured the average time required to monitor a single cell:



The first striking feature of these curves is that, while for simple formulae *base* and *sparse* are significantly faster than *gpu*, for higher number of leaves *base* ends up having average times similar to those of *gpu*. Moreover, *base*'s curve looks steeper than the other two. The same values can be averaged over the number of leaves: that is, given the average time required to monitor a cell when monitoring a formula with n leaves, we can divide that time value by n :



The chart in the previous page shows how the *gpu* curve significantly lowers the time per cell per leaf, when the number of leaves increases. Our explanation for this behaviour is that the GPU implementation suffers from a remarkable overhead in the CPU-GPU communication: this delta becomes less influent when making the formula more complex and thus the matrices bigger. Finally, it is relevant to measure the increase rate of these curves: to do so, we divided all measured times by the first experiment in each curve.



We observed that *base* scales linearly w.r.t. the number of leaves: for instance, the experiment with 32 leaves is 16 times slower than the experiment (with the same implementation) with 2 leaves. The curve for the *sparse* form is more gradual, with the last experiment being ~ 4 times slower than the first one. But the implementation with the smoothest curve is the *gpu*-based one, with the 32-leaves experiment being only ~ 1.5 times slower than the 2-leaves one.

Several other tests, where we made the encoded formula more complex in alternative ways (such as using consecutive *next* operators or interleaving temporal and classical operators), yielded curves following the same patterns. The conclusion of these experiments is that the neural encoding allows for major speed-up and optimisation even with low effort in terms of coding (such is the case for the *sparse* form). The *sparse* implementation proved to be the fastest and it is henceforth the best choice for monitoring *regular* formulae; the *gpu* implementation showed the best scaling factor, thus being the best choice when the encoded property is especially complex.

VI. FUTURE WORK

Besides parallel computation, a key feature of neural networks is the intrinsic learning capability. In our framework, the network encodes a logical property; the supervised learning (observing and learning from traces) corresponds to adapt the encoded formula to the observed patterns.

Although it is usually the case that systems have to be adapted to a given specification, our approach has a number of real-life and research application domains,

such as the Business Process and Multi-Agent Systems areas.

The IEEE Task Force on Process Mining recently released the Process Mining Manifesto [17], defining a set of guiding principles and listing important challenges in the field. The goal is to increase the maturity of process mining as a new tool to improve the (re)design, control, and support of operational business processes. Amongst the announced goals listed in that document, the main one is *to discover, monitor and improve real processes (i.e., not assumed processes) by extracting knowledge from event logs readily available in today's (information) systems*. Therefore, process mining systems aim at closing the gap between the huge (and growing) amount of stored data about processes and activities and the need of making the business process management flexible and competitive for ever-evolving contexts. For instance, when it comes to big companies (such as banks), in some of their branches the task execution often differs from the rigid protocol enforcement, due to obstacles (from broken printers to strikes) or by adaptations to specific needs (dynamic resources reallocation). Hard violations (security, privacy) have to be detected and prevented. On the other hand, soft violations [18] (discrepancies between high-level directives and real implementation) are unavoidable or even necessary to make the system flexible and optimised to real conditions. Thus, the original specification should be adapted in order to include them, and the learned model can be used as a benchmark for new branches. For example, in a bank, the MIFID regulation prescribes that before signing the customer of a financial product should be informed adequately. This is a hard constraint which should not be violated. In contrast, a constraint coming from internal regulations, such as that some document must be sent in paper format rather than scanned, can be violated without major problems, if the scanned version is still compliant with laws. Thus, if a bank branch considered successful and compliant to hard constraints, substituted scanned versions of a given documents to paper ones despite the original specifications, such specifications may be revised. The source of the modification should be the traces of the activity of the successful branch, which can be fed as input to our system, encoding the property to be revised.

Concerning intelligent agents, autonomy is a key factor in the resolution of complex problems and it fosters a flexible behaviour in a number of theoretical scenarios and practical application areas [19]. From the point of view of the designer of a multi-agent system (MAS), autonomy has to be counterbalanced by mechanisms for control and regulation of the agents. In particular, one prominent challenge is how to configure and organise MAS, given their constantly changing structure [20]. One leading solution is to employ the use of social norms. In human societies, social norms are essential to regulation, coordination, and cooperation; normative multi-agent systems [20] combine

social norms and MAS. The standard approach is to monitor the compliance of agents w.r.t. norms, using techniques of sanctioning and norms enforcement. Soft norms implement a more tolerant regulation, including peculiarities as contrary-to-duty, exceptions, and the deontic obligation/permission dualism. Violating norms, and -more generally- displaying a non-scripted behaviour, is a natural consequence of being autonomous. It is worth stressing that there are several reasons that may cause unpredictability in the behaviour of an agent; for instance, in the area of planning, possible causes include: bona-fide deliberation (e.g. if the agent is given a plan based on Disjunctive Temporal Network [21] or over-subscription [22]), external causes (plan failures due to dynamic or non-deterministic environment [23]), or simply malicious behaviour (for agents in a competitive environment, when the intentional violation of norms is used to harm the competing agent(s) [24]). In the traditional normative approach to MAS, agents that do not comply with norms are sanctioned (or have to deal with additional norms), while well-behaving agents are valued according to their performance outcomes [25]. We claim that these approaches do not fully capture nor exploit the richness of behaviours that arise from an open, heterogeneous MAS: in our vision, norms should be adaptive, and they should be refined taking into account the behaviour of agents showing better performance. This approach would allow an open-MAS to implement a task in an autonomous way starting from some abstract task description ϕ , while the designer monitors the execution of all plans in parallel and, if some deviation ϕ_i from the benchmark leads to an improvement in performance (outcome-based evaluation), it has to be learned and used as the starting point for the following iterations.

VII. CONCLUSIONS

First, we developed a runtime monitoring system, RuleRunner. RuleRunner is a rule-based online monitor observing finite but expanding traces and returning an FLTL verdict. Besides the introduction in this paper, we provide a technical report with formal analysis on ArXiv [16] and an online tool (RuleRunner.jnlp). Secondly, we discussed the encoding, partially based on CILP's Translation Algorithm, of a RuleRunner rule system in a standard feedforward neural network with recurrent connections and one hidden layer. Thirdly, we showed how matrix-based optimisations allows for major improvements in terms of performance, either concerning actual speed (matrices in sparse form) or scaling factor (GPU computation). Finally, we discussed how our system is suitable for learning, and mentioned two real-life and research application domains (namely the Business Process and Multi-Agent Systems areas) that would benefit from our approach.

REFERENCES

- [1] IBM, P. Zikopoulos, and C. Eaton, *Understanding Big Data: Analytics for Enterprise Class Hadoop and Streaming Data*, 1st ed. McGraw-Hill Osborne Media, 2011.
- [2] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," *Comput. Netw.*, vol. 54, no. 15, pp. 2787–2805, Oct. 2010.
- [3] Q. V. Le, M. Ranzato, R. Monga, M. Devin, G. Corrado, K. Chen, J. Dean, and A. Y. Ng, "Building high-level features using large scale unsupervised learning," in *ICML*, 2012.
- [4] A. Coates, B. Huval, T. Wang, D. J. Wu, B. C. Catanzaro, and A. Y. Ng, "Deep learning with cots hpc systems," in *ICML (3)*, 2013, pp. 1337–1345.
- [5] A. Pnueli, "The temporal logic of programs," in *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*. Washington, DC, USA: IEEE Computer Society, 1977, pp. 46–57.
- [6] M. Leucker and C. Schallhart, "A brief account of runtime verification," *J. Log. Algebr. Program.*, vol. 78, no. 5, pp. 293–303, 2009.
- [7] —, "A brief account of runtime verification," *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, may/june 2009. [Online]. Available: <http://dx.doi.org/10.1016/j.jlap.2008.08.004>
- [8] O. Lichtenstein, A. Pnueli, and L. D. Zuck, "The glory of the past," in *Logic of Programs*, ser. Lecture Notes in Computer Science, R. Parikh, Ed., vol. 193. Springer, 1985, pp. 196–218.
- [9] A. Bauer, M. Leucker, and C. Schallhart, "The good, the bad, and the ugly, but how ugly is ugly?" in *RV*, ser. Lecture Notes in Computer Science, O. Sokolsky and S. Tasiran, Eds., vol. 4839. Springer, 2007, pp. 126–138.
- [10] —, "Monitoring of real-time properties," in *FSTTCS*, ser. Lecture Notes in Computer Science, S. Arun-Kumar and N. Garg, Eds., vol. 4337. Springer, 2006, pp. 260–272.
- [11] C. Eisner, D. Fisman, J. Havlicek, Y. Lustig, A. McIsaac, and D. V. Camphout, "Reasoning with temporal logic on truncated paths," in *CAV'03*, 2003, pp. 27–39.
- [12] G. G. Towell and J. W. Shavlik, "Knowledge-based artificial neural networks," *Artif. Intell.*, vol. 70, no. 1-2, pp. 119–165, 1994.
- [13] A. S. d'Avila Garcez and G. Zaverucha, "The connectionist inductive learning and logic programming system," *Appl. Intell.*, vol. 11, no. 1, pp. 59–77, 1999.
- [14] S. Hoelldobler and Y. Kalinke, "Towards a new massively parallel computational model for logic programming," in *ECAI94 workshop on Combining Symbolic and Connectionist Processing*, 1994, pp. 68–77.
- [15] H. Barringer, D. E. Rydeheard, and K. Havelund, "Rule systems for run-time monitoring: from eagle to ruler," *J. Log. Comput.*, vol. 20, no. 3, pp. 675–706, 2010.
- [16] A. Perotti, "RuleRunner technical report," *ArXiv e-prints*, Jun. 2013.
- [17] W. M. P. van der Aalst et.al., "Process mining manifesto," in *Business Process Management Workshops (1)*, 2011, pp. 169–194.
- [18] J. Meyer and R. Wieringa, *Deontic logic in computer science: normative system specification*. J. Wiley, 1993.
- [19] M. J. Wooldridge, *An Introduction to MultiAgent Systems (2. ed.)*. Wiley, 2009.
- [20] C. D. Hollander and A. S. Wu, "The current state of normative agent-based systems," *J. Artificial Societies and Social Simulation*, vol. 14, no. 2, 2011.
- [21] P. R. Conrad and B. C. Williams, "Drake: An efficient executive for temporal plans with choice," *J. Artif. Intell. Res. (JAIR)*, vol. 42, pp. 607–659, 2011.
- [22] M. B. Do and S. Kambhampati, "Partial satisfaction (over-subscription) planning as heuristic search," in *In Proceedings of KBCS-04*, 2004.
- [23] R. P. van der Krogt, M. M. de Weerd, N. Roos, and C. Witteveen, "Multiagent planning through plan repair," in *Proceedings of the Fourth International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS-05)*. ACM press, pp. 1337–1338.
- [24] Z. Rui, T. Fu, D. Lai, and Y. Jiang, "Cooperation among malicious agents: a general quantitative congestion game framework," in *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems - Volume 3*, ser. AAMAS '12, 2012, pp. 1331–1332.
- [25] L. Laurian, J. Crawford, M. Day, P. Kouwenhoven, G. Mason, N. Ericksen, and L. Beattie, "Evaluating the outcomes of plans: theory, practice, and methodology," *Environment and Planning B: Planning and Design*, vol. 37, no. 4, pp. 740–757, 2010.